

Network Device Alert Investigation Agent

ANONYMOUS AUTHOR(S)

ACM Reference Format:

Anonymous Author(s). 2026. Network Device Alert Investigation Agent. 1, 1 (September 2026), 12 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

In recent years, the rapid expansion and increasing complexity of enterprise network infrastructures have made alert fatigue and information overload persistent bottlenecks for Security Operations Centers (SOCs). Large-scale empirical measurements and practitioner studies have shown that operational SOCs must process substantial volumes of security alarms, many of which correspond to false positives, benign triggers, or failed attack attempts rather than true attacks [1, 13, 17, 30]. In particular, heterogeneous network devices and security sensors, such as IDS, IPS, and WAFs, continuously emit low-level alerts that are often analyzed independently, although they may be logically connected to the same intrusion [25]. For sophisticated multi-stage attacks, the evidence of reconnaissance, exploitation, command-and-control, lateral movement, and data exfiltration is often dispersed across multiple devices, assets, network regions, and time windows [23]. As a result, the critical clues required to characterize an attack chain are fragmented and buried in massive noisy alert streams. How to suppress redundant and low-value alerts while extracting, correlating, and explaining the alert context associated with a specific real attack has therefore become a core bottleneck for intelligent SOC operations, attack provenance reconstruction, and timely incident response.

Despite extensive academic and industrial efforts in alert correlation and threat investigation, existing methods remain inadequate for investigating large volumes of heterogeneous network device alerts in enterprise SOCs. Traditional techniques based on rules, statistical features, and learning models reduce alert volume and infer attack scenarios through expert designed templates, security event prediction, and information flow correlation [23, 28]. However, these techniques often depend on predefined correlation patterns or historical regularities, and thus become fragile in operational SOC environments where benign triggers, failed attack attempts, and false positives dominate alert streams [1, 30]. More recently, provenance graph based approaches have become an important direction for attack reconstruction. By modeling causal dependencies among system entities and applying graph analysis or representation learning, they can recover compact attack narratives from large scale audit logs and EDR telemetry [2, 4, 5, 7–9, 12, 15, 19, 29]. Nevertheless, most existing provenance systems are designed around host execution semantics, including processes, files, sockets, system calls, and application layer events. As a result, these methods provide limited support for semantic alignment and attack level correlation across massive alerts generated by heterogeneous network devices. Meanwhile, recent LLM and agent assisted systems have explored guided response, alert triage, and alert graph reasoning in SOC workflows [3, 10, 11]. However, they are still largely focused on static assistance or individual reasoning tasks, while their

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/9-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

reliability is further challenged by fragmented long context evidence and hallucination risks [14]. These limitations motivate us to formulate network device alert correlation as a stateful, hypothesis driven, and agentic investigation problem, rather than as a conventional single pass correlation or summarization task.

To bridge the gap, we propose a network device alert investigation agent that turns noisy and fragmented network device alerts into evidence-grounded investigation narratives. Our design consists of three key components: a hierarchical multi-agent workflow that decomposes alert investigation into coordinated reasoning and verification steps; a working memory module that maintains the evolving investigation context across long and fragmented alert streams; and an experience reuse mechanism that retrieves and adapts prior investigation workflows and historical cases to new alert scenarios. We next describe the technical challenges and key insights in building such an agentic investigation system.

Technical Challenges. Building a reliable agentic investigation system for heterogeneous network-device alerts presents three technical challenges:

- ① *How to orchestrate hierarchical and multi-agent investigation workflows over noisy, heterogeneous, and weakly connected alerts?* Network-device alerts are produced by diverse security sensors, such as IDS, IPS, WAFs, firewalls, and traffic analyzers, each of which follows different rule semantics, severity definitions, and reporting granularities. Unlike host-level provenance traces, network alerts often lack explicit causal dependencies and may provide only partial evidence about attack intent, execution outcome, and victim impact. Therefore, a single-pass correlation or monolithic LLM reasoning process is insufficient to reliably distinguish true attacks from benign triggers, failed attempts, and redundant alerts, which frequently coexist in operational SOC environments [1, 30]. The challenge is to decompose the investigation into specialized but coordinated subtasks, such as alert triage, context expansion, hypothesis generation, evidence checking, and report construction, while preventing local reasoning errors from propagating across agents.
- ② *How to maintain faithful investigation context under long, fragmented, and evolving evidence?* Network alert investigation is inherently stateful: evidence relevant to the same intrusion may be scattered across multiple devices, assets, network zones, and time windows. However, LLMs do not always use long contexts reliably, and relevant evidence can be overlooked when it is buried in the middle of lengthy inputs [20]. Simply appending all retrieved alerts to the prompt is also impractical, because redundant alerts dilute important clues and token limits may force the model to discard useful historical context. The challenge is to design a working memory that compactly records seed alerts, related entities, temporal constraints, intermediate hypotheses, supporting and contradicting evidence, and unresolved questions. More importantly, the memory should distinguish verified facts from tentative assumptions, so that the agent can update its investigation state without introducing unsupported conclusions or hallucinated attack narratives [14, 26].
- ③ *How to reuse historical investigation experience without overfitting to stale or case-specific patterns?* SOC investigations are repetitive in procedure but diverse in manifestation. Similar attack behaviors may trigger different alerts across vendors, rule versions, network regions, and asset configurations, while superficially similar alerts may correspond to different outcomes, such as successful compromise, failed exploitation, benign scanning, or policy violation. Existing rule-based and template-based correlation methods can encode expert knowledge, but they often become brittle when alert semantics deviate from predefined patterns [23, 25, 28]. The challenge is therefore to reuse prior workflows and cases as adaptable investigation experience rather than rigid decision rules. The agent must retrieve semantically relevant historical cases,

abstract reusable investigation steps, adapt them to the current alert context, and verify their applicability against fresh evidence, so that experience reuse improves investigation efficiency without sacrificing correctness [27].

Key Insights. To address these challenges, we incorporate several key insights and techniques:

- *Separate investigation responsibilities and place an evidence gate between reasoning and conclusions.* We divide the investigation among a Planner, Executor, Expert, and Verifier instead of relying on a monolithic agent. The Planner maintains the global investigation structure, the Executor performs bounded tool actions, the Expert interprets returned observations, and the Verifier accepts a finding only when it is supported by evidence from the current case. This role separation limits the propagation of local reasoning errors while preserving coordinated, end-to-end investigation (§ 3.1).
- *Represent evolving investigation context as structured state rather than a growing transcript.* We organize working memory as an Investigation Task Tree and an entity-and-evidence table. The tree separates the global goal, investigation branches, and executable atomic tasks, while explicit task states expose the current frontier and retain failed paths for refinement or backtracking. In parallel, unified entity identifiers and evidence references deduplicate observations across data sources and preserve their provenance, allowing the agent to distinguish unresolved hypotheses from evidence-supported findings (§ 3.3).
- *Separate environment knowledge, case experience, and reusable procedures in long-term memory.* We construct a hierarchical memory comprising semantic memory for telemetry schemas, episodic memory for verified historical investigation traces and recovery paths, and procedural memory for investigation skills abstracted from related episodes. These layers are retrieved according to the current investigation stage and adapted to the active entities and data environment. Crucially, historical outcomes guide where and how to investigate but never substitute for fresh evidence about the current alert (§ 3.2).

Evaluation.

2 BACKGROUND

2.1 Network Security Incident Investigation

In an enterprise Security Operations Center (SOC), network security incident investigation typically begins with an alert generated by an IDS, IPS, WAF, firewall, or other monitoring device and proceeds through triage, evidence collection, analysis, and escalation or closure [16, 24]. Because an alert only indicates that a detector observed activity matching a rule, signature, or behavioral condition, analysts must retrieve its surrounding context before judging whether an incident occurred. In practice, they pivot from the initial alert to temporally adjacent alerts, source and destination entities, affected services and assets, network zones, and observations from other security devices. This process is necessary because operational alert streams contain benign triggers and unsuccessful attack attempts as well as alerts associated with successful attacks [1, 30]. The essence of enterprise SOC investigation is therefore alert-context correlation: analysts connect heterogeneous observations across devices, entities, and time windows, determine whether the resulting relationships form a coherent attack scenario, and retain the evidence needed to justify the final assessment [25]. Unlike simple aggregation based on shared fields, an investigation must also distinguish supporting from contradicting evidence and revise its interpretation when newly retrieved context does not support the current hypothesis.

2.2 Large Language Models for Cybersecurity

Large language models (LLMs) have been applied to a range of cybersecurity tasks because they can interpret technical text, generate structured outputs, and interact with external tools [31]. Representative applications include analyzing vulnerability reports and patches in vulnerability management [21], deriving protocol information to guide network-protocol fuzzing [22], and planning multi-step penetration-testing activities while interpreting tool outputs [6]. LLMs have also been studied for incident-report summarization and for SOC agents that retrieve logs, enrich alerts, analyze evidence, and generate investigation reports through existing security tools [3, 18]. These applications suggest that LLMs can serve as a reasoning and orchestration layer over heterogeneous security data and tools. However, their use in security-critical workflows remains constrained by unreliable use of long contexts and the risk of producing unsupported statements [14, 20]. Consequently, an LLM-based security system should ground its conclusions in retrieved evidence and retain explicit investigation state rather than relying on a single unconstrained model response.

3 METHODOLOGY

In this section, we present our memory-augmented multi-agent framework for investigating heterogeneous network-device alerts. As shown in ??, the framework conducts an iterative, evidence-grounded investigation through three cooperating components. First, a multi-agent workflow separates global planning, tool execution, evidence analysis, and result verification into specialized roles. Second, a working memory organizes the investigation goal, active branches, atomic tasks, discovered entities, and supporting evidence, allowing the system to preserve progress and revise unsuccessful paths. Third, a hierarchical long-term memory provides knowledge of the current data environment, prior investigation episodes, and reusable investigation procedures to guide planning and recovery.

3.1 Evidence-Grounded Multi-Agent Investigation

Reliable investigation requires both local evidence exploration and global control over the evolving reasoning process. A monolithic agent must simultaneously plan the investigation, construct queries, interpret their outputs, and decide whether the available evidence supports a conclusion. Errors made in one of these activities can therefore influence all subsequent decisions. We instead divide the investigation among four specialized agents: a *Planner*, an *Executor*, an *Expert*, and a *Verifier*. The agents communicate through the shared working memory, while the long-term memory provides environment knowledge and reusable experience. Importantly, retrieved experience is used only to guide the investigation. Every finding about the current case must remain traceable to evidence returned by the current investigation.

Planner. The Planner is responsible for global task decomposition and maintenance of the investigation plan. It receives the investigation goal, the seed-alert context, and the current working-memory state, and then organizes the goal into candidate investigation branches and executable atomic tasks. During initialization, the Planner may use relevant procedures and similar historical episodes to determine which entities and data sources should be examined first. As the investigation progresses, it selects the next task according to the completed tasks, available evidence, unresolved questions, and feedback from the Verifier. When a query fails, returns excessive noise, or does not support the current direction, the Planner can refine the task, activate an alternative branch, or backtrack to an earlier decision. The Planner does not directly execute tools or treat a planned relation as an established fact.

Executor. The Executor converts one active atomic task into a concrete query or tool invocation. It uses the relevant environment and schema knowledge to identify valid data sources, fields, and query

constraints, and then executes the resulting action against the available security telemetry. Each execution produces a structured record containing the performed action and its raw observation. Successful query results, empty results, execution errors, and overly broad result sets are all preserved as distinct outcomes because each can affect the next investigation decision. The Executor reports what the tool returned but does not infer the security meaning of the observation or decide whether the investigation goal has been satisfied.

Expert. The Expert interprets the Executor’s observation in the context of the active task. It extracts security-relevant entities and relationships, connects them to their originating records, and determines whether the observation supports the current hypothesis, contradicts it, or provides only a clue for further investigation. The Expert produces structured candidate findings together with their supporting evidence and identifies new questions exposed by the result. For example, an empty query may indicate that the current pivot is unsuitable, whereas a schema error may reveal that the task should be reformulated using another field. These interpretations are recorded as updates to the working memory, but they remain provisional until they pass verification.

Verifier. The Verifier provides global supervision over the task tree, accumulated evidence, and candidate findings. It checks whether each finding is supported by evidence from the current case, whether the claimed relationship is consistent with the retrieved records, and whether contradictory observations or unresolved branches have been overlooked. A finding that relies only on a historical episode, a planned relation, or a weak contextual coincidence is not accepted as case evidence. Based on this assessment, the Verifier can confirm a finding, request additional evidence, reject an unsupported interpretation, or instruct the Planner to revise or backtrack the investigation. The investigation is allowed to terminate only when the goal has been addressed and the accepted findings are sufficiently supported. Consequently, only verified findings are incorporated into the final investigation narrative, providing an explicit evidence gate between multi-step agent reasoning and the reported conclusion.

3.2 Hierarchical Long-Term Memory

Enterprise alert investigation depends on two forms of knowledge that are difficult to preserve in a single prompt: knowledge of the current telemetry environment and experience from previous investigations. We therefore introduce a unified Hierarchical Investigation Memory (HIM) consisting of semantic, episodic, and procedural memory, as illustrated in [Figure 1](#). Semantic memory describes what data are available and how they can be queried; episodic memory records how individual historical cases were investigated; and procedural memory abstracts recurring investigation strategies from multiple episodes. This hierarchy separates environment facts, case-specific experience, and reusable methods, allowing each agent to retrieve only the type of knowledge required by its current task. Historical memory is treated as guidance rather than evidence: any claim about the active alert must still be established from telemetry retrieved during the current investigation.

3.2.1 Semantic Memory. Semantic memory provides a structured description of the investigation environment. It is constructed from database catalogs and system tables, schema interfaces exposed by security platforms, data dictionaries maintained for log pipelines, product documentation, and bounded read-only profiling of low-cardinality fields. After normalization, this information is represented as a graph containing *Environment*, *Table*, *Field*, and *EnumValue* nodes. Relations such as *HAS_TABLE*, *HAS_FIELD*, and *HAS_ENUMVALUE* connect these nodes and preserve the provenance of each schema fact. Field descriptions additionally record security semantics when available, including the represented entity type, temporal meaning, and whether the field can be used to join records across data sources. This representation enables the Planner to identify telemetry relevant



fig/LTM.pdf

Fig. 1. Overview of the Hierarchical Investigation Memory (HIM). Semantic memory models the security-data environment, episodic memory retains verified investigation traces and recovery paths, and procedural memory abstracts reusable investigation skills.

to an investigation branch and helps the Executor construct valid queries without repeatedly rediscovering schemas. For example, when a task requires pivoting from a source IP address to authentication activity, semantic memory can locate candidate identity tables, their address and timestamp fields, and the values used to distinguish successful and failed logins. Because deployments evolve, retrieved schema knowledge is checked against the current environment before execution, and invalid or outdated entries are reported as execution outcomes rather than silently assumed to remain correct.

3.2.2 Episodic Memory. Episodic memory preserves concrete investigation experience at the case level. A raw investigation trace is represented as

$$\tau_i = \left(c_i, q_i, \left\langle h_{i,j}, a_{i,j}, o_{i,j} \right\rangle_{j=1}^{m_i}, y_i, v_i \right), \quad (1)$$

where c_i is the initial alert context, q_i is the investigation goal, $h_{i,j}$ denotes the intent of step j , $a_{i,j}$ is the corresponding query or tool action, $o_{i,j}$ is its observation, y_i is the final outcome, and v_i records the verification status. The trace is cleaned, desensitized, and transformed into an episode graph with *Episode*, *Step*, *Evidence*, and *Outcome* nodes. Directed relations encode step order, evidence production, and the connection between a trace and its final outcome. Each episode also maintains a compact episode card containing its initial context, core investigation path, verified outcome, and key lesson. We retain useful recovery paths within successful traces, because an empty result, an invalid query, or an overly broad search can reveal how the investigation was redirected. To control memory quality and prevent evaluation leakage, the memory builder admits only verified traces from the designated training corpus that contain a submitted final answer and at least one executed query; evaluation cases are never used to construct or update this memory. Retrieved episodes therefore offer examples of how related investigations proceeded, but their entities and outcomes are not copied into the current case.

3.2.3 Procedural Memory. While episodic memory is specific to individual cases, procedural memory captures investigation strategies that recur across similar cases. Procedures are derived by grouping verified episodes according to their goals, initial entities, investigation phases, data-source usage, and failure-recovery patterns. Let $\mathcal{E}$ denote the episode collection and μ_k the prototype of

procedure group k . The associated episode cluster is

$$C_k = \{E_i \in \mathcal{E} \mid \text{sim}(E_i, \mu_k) \geq \delta\}, \quad (2)$$

where the similarity function combines the preceding structural and semantic dimensions, and δ is the grouping threshold. Within each cluster, steps are aligned by investigation phase rather than by surface query text. The system then extracts a high-frequency main path, useful conditional branches, recurring recovery actions, and the conditions under which an investigation can terminate. Semantic memory is used during this abstraction to replace environment-specific table names, field names, and indicators with roles such as *authentication record*, *source entity*, or *time constraint*. The resulting procedure contains a trigger, an ordered set of investigation steps, conditional branches for empty, noisy, or erroneous observations, and explicit verification and stopping conditions. Each procedure remains linked to the episodes from which it was abstracted, enabling the agents to inspect a concrete example when the generalized instruction is insufficient.

3.2.4 Long-Term Memory Construction and Storage. HIM is constructed offline in three stages. First, the semantic-memory builder enumerates the available security-data environments, tables, fields, and low-cardinality values from the sources described above. It normalizes their names and descriptions, assigns security semantics to fields, and stores the resulting nodes and relations in a schema graph. Second, the episodic-memory builder ingests completed investigation traces and maps each trace to the representation in Equation 1. Traces without a verified outcome or an executed query are discarded, and only cases from the designated training corpus are admitted. The retained traces are desensitized and converted into episode graphs and compact episode cards. Successful recovery paths are preserved together with the main path so that query failures and subsequent corrections remain available as investigation experience. Third, the procedural-memory builder groups verified episodes according to Equation 2, aligns their steps by investigation phase, and extracts recurring main paths, conditional recovery branches, verification rules, and stopping conditions. During this abstraction, semantic memory replaces case-specific tables, fields, and indicators with their functional roles, producing reusable investigation-skill files rather than copies of historical queries. Each memory item retains its source and construction metadata so that affected semantic entries and procedures can be refreshed when the telemetry environment changes. This process converts heterogeneous environment descriptions and historical traces into three complementary memory layers while maintaining a clear separation between training experience and evaluation cases.

The three memory layers are persisted in representations that match their respective structures. Semantic memory is stored as a property graph whose *Environment*, *Table*, *Field*, and *EnumValue* nodes are connected by schema relations, allowing an agent to traverse from a data environment to queryable fields and their meanings. Episodic memory is stored as episode-centered subgraphs containing ordered *Step*, *Evidence*, and *Outcome* nodes; a compact episode card is maintained alongside each subgraph to summarize its initial context, core path, verified result, and recovery lesson. Procedural memory is stored as structured investigation-skill files that explicitly encode the trigger, required inputs, main investigation path, conditional recovery branches, verification rules, and stopping conditions. Each skill remains connected to its source episodes through ABSTRACTEDFROM relations, while all three layers retain provenance and construction metadata. This organization preserves the distinct granularity of schema facts, case histories, and reusable procedures while allowing their relationships to be followed across the memory hierarchy.

3.3 Investigation Task Tree as Working Memory

A network-alert investigation may span multiple data sources, hypotheses, and recovery attempts. As the investigation grows, a flat interaction history does not explicitly distinguish completed work from unresolved questions, nor does it provide a stable index from a reasoning step to its supporting records. We therefore maintain a case-specific working memory that separates investigation control from accumulated observations. At investigation step t , the working-memory state is defined as

$$WM_t = \langle T_t, R_t \rangle, \quad (3)$$

where T_t is an Investigation Task Tree that records the decomposition and execution state of the current investigation, and R_t is an entity-and-evidence table that indexes the entities and observations discovered so far. The two structures are updated throughout one case and provide the shared state from which subsequent investigation decisions are made.

Investigation Task Tree. The task tree T_t is a strict rooted tree with three semantic levels. The root at level $L0$ represents the global investigation goal derived from the seed alert, such as determining whether the alert belongs to a broader security incident. Level $L1$ contains investigation branches that divide this goal into coherent subgoals. A branch may focus on an involved entity, a hypothesized relationship, or a class of telemetry that can confirm or reject part of the investigation. These branches represent questions to be resolved rather than conclusions already supported by evidence. Level $L2$ contains atomic tasks that can be translated into a bounded query or tool action. Each atomic task specifies a concrete information need and the entities required to execute it, such as retrieving records associated with an address within the relevant time range. Only atomic tasks are directly executable; higher-level nodes organize their purpose and dependencies.

The parent-child relation records why each task is required. A path from the root to a leaf therefore connects a concrete tool action to the branch it serves and, ultimately, to the global investigation goal. This constraint prevents isolated queries from being executed without an explicit investigative purpose. Each node has exactly one parent, except for the root, so the control structure remains unambiguous even when several branches refer to the same host, address, or user. Shared entities are not duplicated as tree nodes. Instead, task nodes reference the corresponding `entity_id` in R_t , allowing the same entity to participate in multiple branches while each task retains a single position in the tree.

Every task node is assigned one of four states: TODO, ACTIVE, DONE, or FAILED. A newly created task is marked TODO; the task currently selected for execution becomes ACTIVE. A task becomes DONE when its requested action has been executed and its observation has been recorded, including cases in which an empty result provides meaningful negative evidence. By contrast, FAILED indicates that the current task formulation cannot address its information need, for example because the selected field is invalid, the action cannot be executed, or the scope remains unusably broad. Importantly, failed tasks are retained rather than removed. Their attempted actions and outcomes explain why a branch was revised and prevent an unsuccessful path from being silently repeated. The status distribution over the tree exposes the current investigation frontier: active and pending leaves indicate remaining work, completed leaves preserve progress, and failed leaves identify locations from which an alternative task or branch may be required.

The tree supports four structural changes as evidence accumulates. *Decomposition* creates the initial $L1$ branches and their first atomic tasks from the global goal. *Expansion* adds a new child task when an observation exposes a follow-up question within the same branch. *Refinement* creates a revised atomic task when the previous formulation is invalid, empty, or excessively noisy; the original node is preserved so that the revision remains traceable. *Backtracking* returns attention to the nearest unresolved ancestor or sibling branch when the current path cannot advance. These

operations allow T_t to grow incrementally without rewriting the full investigation plan, while the fixed hierarchy keeps global goals, investigation branches, and executable actions clearly separated.

Entity and Evidence Table. The table R_t complements the task tree by storing what the investigation has observed. Each discovered entity is assigned a unified `entity_id` together with its normalized type and value, the task that introduced it, and references to the observations in which it appears. Evidence entries preserve the originating data source, the executed action, and the returned record or its raw-result reference. When the same entity is recovered from different tasks or telemetry sources, the existing identifier is reused and the new evidence reference is attached to it. This deduplication allows later tasks to pivot across data sources without creating disconnected copies of the same investigative object.

Task nodes store references to entities and evidence in R_t rather than embedding raw telemetry in the tree. Consequently, T_t remains a compact control structure, while R_t maintains the provenance needed to inspect or verify an observation. Candidate relationships and findings are likewise linked to their supporting evidence references, so their presence in working memory does not by itself make them accepted conclusions. Together, the task tree and the entity-and-evidence table preserve the complete state of the active case: T_t records what has been attempted and what remains unresolved, whereas R_t records which entities and observations justify subsequent reasoning. The detailed sequence by which the agents update this state is described in Section 3.4.

3.4 End-to-End Investigation Workflow

Given a seed alert and access to enterprise security telemetry, the framework executes the investigation as a closed loop over planning, evidence acquisition, interpretation, and verification. The Investigation Task Tree provides the control state of this loop, the entity-and-evidence table preserves the observations produced by each step, and the long-term memory supplies environment knowledge and reusable experience when needed. The workflow continues until the global investigation goal is supported by verified findings or the remaining unresolved branches cannot be advanced with the available data.

Initialization. The framework first creates the initial working memory WM_0 . The seed alert becomes the L0 investigation goal in the task tree, while the alert record and its extracted entities are inserted into the entity-and-evidence table. The Planner then performs stage-aware retrieval from the long-term memory. Semantic memory provides the tables, fields, entity types, and temporal attributes relevant to the seed context. Procedural memory contributes an investigation skill whose trigger and required inputs match the current alert, and episodic memory may provide a small number of verified traces with similar goals or initial entities. The Planner uses this information to create the initial L1 investigation branches and their L2 atomic tasks. Retrieved procedures and episodes determine possible directions of inquiry, but their historical outcomes are not inserted into the current evidence table.

Task Planning. At each iteration, the Planner reads the current task tree, accumulated entities and evidence, and any unresolved verification requests. It selects an eligible `TODO` leaf and marks it `ACTIVE`. If the existing leaves do not address an unresolved question, the Planner expands the corresponding L1 branch with a new atomic task. Each selected task specifies the information to be obtained and references the entities required for its execution. The Planner may consult the current step of the retrieved procedure to determine the next investigation action, while related episodes are used only when a concrete example is needed to disambiguate the procedure. The resulting task must remain grounded in the current working-memory context and be executable as a bounded query or tool invocation.

Evidence Acquisition and Interpretation. The Executor receives the active task, its referenced entities, and the relevant semantic-memory entries. It resolves the required data source and fields,

constructs the query or tool action, and executes it against the available telemetry. The executed action and raw observation are stored in the entity-and-evidence table regardless of whether the result contains matching records, is empty, is excessively broad, or reports an execution error. The Expert then interprets the observation with respect to the active task. It normalizes newly discovered entities, reuses existing `entity_id` values when possible, and records candidate relationships and findings together with their supporting evidence references. An observation may support the current direction, contradict an earlier candidate finding, or expose a new question. In the latter case, the question is attached to the appropriate task-tree branch so that it can be decomposed into a subsequent atomic task. Once the action and its outcome have been recorded, the active task is marked DONE; if the action cannot address the task under its current formulation, it is marked FAILED and retained with its execution record.

Verification and Recovery. The Verifier evaluates candidate findings against the current task tree and evidence table. It checks whether the claimed entities and relationships are present in the retrieved records, whether the cited observations address the corresponding task, and whether conflicting evidence or unresolved branches have been overlooked. A supported finding is accepted for the current case, whereas an unsupported or incomplete finding is returned to the Planner with a specific evidence requirement. The subsequent recovery action depends on the observed failure mode. An empty result can trigger a broader time range or an alternative entity pivot; a noisy result can lead to a narrower scope or an additional filter; and a schema or execution error can trigger renewed semantic-memory lookup and query reformulation. When the generalized recovery instruction is insufficient, the Planner may retrieve an episode containing a similar failure-and-recovery pattern. If the active branch cannot advance, the task tree preserves the failed path and control backtracks to the nearest unresolved ancestor or sibling branch. In every case, a recovery action is represented as a new task, ensuring that the original attempt and the reason for revision remain auditable.

Termination and Reporting. After each verification step, the Verifier assesses whether the investigation can terminate. The global goal is considered addressed only when the relevant L1 branches have been resolved or explicitly identified as unsupported by the available telemetry, accepted findings are backed by current-case evidence, material contradictions have been examined, and the applicable procedural verification and stopping conditions have been satisfied. If any of these conditions is unmet, the Verifier returns the missing question or evidence requirement to the Planner and the loop continues. Otherwise, the framework constructs the final investigation narrative from the accepted findings, their task-tree paths, and their evidence references. Each reported conclusion is linked to the observations that support it, while unsuccessful paths and remaining evidence gaps are retained as investigation limitations. Thus, long-term memory guides how the system investigates, working memory records what occurred during the current investigation, and only verified current-case evidence determines the reported outcome.

4 EVALUATION

5 RELATED WORK

6 CONCLUSION

REFERENCES

- [1] Bushra A Alahmadi, Louise Axon, and Ivan Martinovic. 2022. 99% false positives: A qualitative study of {SOC} analysts' perspectives on security alarms. In *31st USENIX Security Symposium (USENIX Security 22)*. 2783–2800.
- [2] Abdullellah Alsaheel, Yuhong Nan, Shiqing Ma, Le Yu, Gregory Walkup, Z Berkay Celik, Xiangyu Zhang, and Dongyan Xu. 2021. {ATLAS}: A sequence-based learning approach for attack investigation. In *30th USENIX security symposium (USENIX security 21)*. 3005–3022.
- [3] Kritan Banstola, Faayed Al Faisal, and Xinming Ou. 2026. Experiences of Using Agentic AI to Fill Tooling Gaps in a Security Operations Center. In *Workshop on Security Operation Center Operations and Construction (WOSOC 2026)*. <https://doi.org/10.14722/wosoc.2026.23016>
- [4] Tristan Bilot, Baoxiang Jiang, Zefeng Li, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, and Thomas Pasquier. 2025. Sometimes Simpler is Better: A Comprehensive Analysis of {State-of-the-Art} {Provenance-Based} Intrusion Detection Systems. In *34th USENIX Security Symposium (USENIX Security 25)*. 7193–7212.
- [5] Zijun Cheng, Qiujian Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. 2024. Kairos: Practical intrusion detection and investigation using whole-system provenance. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3533–3551.
- [6] Gelei Deng, Yi Liu, Victor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 847–864. <https://www.usenix.org/conference/usenixsecurity24/presentation/deng>
- [7] Feng Dong, Shaofei Li, Peng Jiang, Ding Li, Haoyu Wang, Liangyi Huang, Xusheng Xiao, Jiedong Chen, Xiapu Luo, Yao Guo, et al. 2023. Are we there yet? an industrial viewpoint on provenance-based endpoint detection and response tools. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2396–2410.
- [8] Feng Dong, Liu Wang, Xu Nie, Fei Shao, Haoyu Wang, Ding Li, Xiapu Luo, and Xusheng Xiao. 2023. {DISTDET}: A {Cost-Effective} distributed cyber threat detection system. In *32nd USENIX Security Symposium (USENIX Security 23)*. 6575–6592.
- [9] Pengcheng Fang, Peng Gao, Changlin Liu, Erman Ayday, Kangkook Jee, Ting Wang, Yanfang Fanny Ye, Zhuotao Liu, and Xusheng Xiao. 2022. {Back-Propagating} system dependency impact for attack investigation. In *31st USENIX security symposium (USENIX Security 22)*. 2461–2478.
- [10] Scott Freitas, Jovan Kalajdjieski, Amir Gharib, and Robert McCann. 2025. AI-driven guided response for security operation centers with Microsoft Copilot for Security. In *Companion Proceedings of the ACM on Web Conference 2025*. 191–200.
- [11] Yiru Gong, Song Liu, Changzhi Zhao, Junrong Liu, Tian Tian, Xiaobo Yang, Bo Jiang, and Zhigang Lu. 2026. GARNET: GoT-Based Alert Reduction and Narrative Event Tracing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 327–335.
- [12] Xueyuan Han, Thomas Pasquier, Adam Bates, James Mickens, and Margo Seltzer. 2020. Unicorn: Runtime provenance-based detector for advanced persistent threats. *arXiv preprint arXiv:2001.01525* (2020).
- [13] Wajih Ul Hassan, Shengjian Guo, Ding Li, Zhengzhang Chen, Kangkook Jee, Zhichun Li, and Adam Bates. 2019. Nodoze: Combatting threat alert fatigue with automated provenance triage. In *network and distributed systems security symposium*.
- [14] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* 43, 2, Article 42 (2025), 55 pages. <https://doi.org/10.1145/3703155>
- [15] Baoxiang Jiang, Tristan Bilot, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, Shahrear Iqbal, Xueyuan Han, and Thomas Pasquier. 2025. {ORTHURUS}: Achieving High Quality of Attribution in Provenance-based Intrusion Detection Systems. In *34th USENIX Security Symposium (USENIX Security 25)*. 7173–7192.
- [16] Leon Kersten, Tom Mulders, Emmanuele Zamboni, Chris Snijders, and Luca Allodi. 2023. “Give Me Structure”: Synthesis and Evaluation of a (Network) Threat Analysis Process Supporting Tier 1 Investigations in a Security Operation Center. In *Nineteenth Symposium on Usable Privacy and Security (SOUPS 2023)*. USENIX Association, 97–111. <https://www.usenix.org/conference/soups2023/presentation/kersten>
- [17] Faris Bugra Kokulu, Ananta Soneji, Tiffany Bao, Yan Shoshitaishvili, Ziming Zhao, Adam Doupe, and Gail-Joon Ahn. 2019. Matched and mismatched SOC: A qualitative study on security operations center issues. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*. 1955–1970.
- [18] Diana Kramer, Lambert Rosique, Ajay Narotam, Elie Bursztein, Patrick Gage Kelley, Kurt Thomas, and Allison Woodruff. 2025. Integrating Large Language Models into Security Incident Response. In *Twenty-First Symposium on Usable Privacy and Security (SOUPS 2025)*. USENIX Association, 133–148. <https://www.usenix.org/conference/soups2025/presentation/kramer>

- [19] Shaofei Li, Feng Dong, Xusheng Xiao, Haoyu Wang, Fei Shao, Jiedong Chen, Yao Guo, Xiangqun Chen, and Ding Li. 2023. Nodlink: An online system for fine-grained apt attack detection and investigation. *arXiv preprint arXiv:2311.02331* (2023).
- [20] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics* 12 (2024), 157–173.
- [21] Peiyu Liu, Junming Liu, Lirong Fu, Kangjie Lu, Yifan Xia, Xuhong Zhang, Wenzhi Chen, Haiqin Weng, Shouling Ji, and Wenhai Wang. 2024. Exploring ChatGPT’s Capabilities on Vulnerability Management. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 811–828. <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-peiyu>
- [22] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. 2024. Large Language Model Guided Protocol Fuzzing. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*. <https://doi.org/10.14722/ndss.2024.24556>
- [23] Sadeqh M Milajerdi, Rigel Gjomemo, Birhanu Eshete, Ramachandran Sekar, and VN Venkatakrishnan. 2019. Holmes: real-time apt detection through correlation of suspicious information flows. In *2019 IEEE symposium on security and privacy (SP)*. IEEE, 1137–1152.
- [24] Alexander Nelson, Sanjay Rekhi, Murugiah Souppaya, and Karen Scarfone. 2025. *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile*. Technical Report NIST Special Publication 800-61 Revision 3. National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-61r3>
- [25] Peng Ning, Yun Cui, and Douglas S Reeves. 2002. Constructing attack scenarios through correlation of intrusion alerts. In *Proceedings of the 9th ACM Conference on Computer and Communications Security*. 245–254. <https://doi.org/10.1145/586110.586144>
- [26] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*. 1–22.
- [27] Avi Shaked, Yulia Cherdantseva, and Pete Burnap. 2022. Model-based incident response playbooks. In *Proceedings of the 17th International Conference on Availability, Reliability and Security*. 1–7.
- [28] Yun Shen, Enrico Mariconti, Pierre Antoine Vervier, and Gianluca Stringhini. 2018. Tiresias: Predicting security events through deep learning. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 592–605.
- [29] Zhiqiang Xu, Pengcheng Fang, Changlin Liu, Xusheng Xiao, Yu Wen, and Dan Meng. 2022. Depcomm: Graph summarization on system audit logs for attack investigation. In *2022 IEEE symposium on security and privacy (SP)*. IEEE, 540–557.
- [30] Limin Yang, Zhi Chen, Chenkai Wang, Zhenning Zhang, Sushruth Booma, Phuong Cao, Constantin Adam, Alexander Withers, Zbigniew Kalbarczyk, Ravishankar K Iyer, and Gang Wang. 2024. True attacks, attack attempts, or benign triggers? an empirical measurement of network alerts in a security operations center. In *33rd USENIX Security Symposium (USENIX Security 24)*. 1525–1542.
- [31] Jie Zhang, Haoyu Bu, Hui Wen, Yongji Liu, Haiqiang Fei, Rongrong Xi, Lun Li, Yun Yang, Hongsong Zhu, and Dan Meng. 2025. When LLMs Meet Cybersecurity: A Systematic Literature Review. *Cybersecurity* 8, 1 (2025), 55. <https://doi.org/10.1186/s42400-025-00361-w>